

HRMS-后端-用户模块

变更记录

记录每次修订的内容，方便追溯。

日期	版本	修订说明	作者
2026-07-13	1.0	初稿，根据实际代码实现编写	-

项目背景

对本次项目的背景以及目标进行描述，方便开发者理解需求，对齐上下文。

本模块来源于 HRMS（人资管理系统）产品规格说明书中第 2 部分——用户管理。当前系统需要一个统一的用户身份管理系统，支持用户注册、登录、登出、角色权限控制，以及管理员的用户CRUD操作。本模块是整个系统的认证与授权基础，所有业务模块均依赖此模块提供登录态验证和角色鉴权。

相关资料

- [人资管理系统（HRMS）详细产品规格说明书](#)

参与人

项目负责人	...
产品经理	...
设计师	...
工程师	...

功能模块

描述用户模块涉及的功能与场景。

本模块核心功能包括：

- 用户注册**：用户自主注册账号（账号>=4位、密码>=8位、确认密码一致、账号不重复）
- 用户登录**：账号+密码登录，MD5加盐加密验证，登录成功记录登录日志，登录失败（密码错误且账号存在）也记录日志
- 用户登出**：清除 session 登录态
- 获取当前用户**：从 session 获取当前登录用户信息，返回脱敏后的 LoginUserVO
- 用户管理（管理员）**：创建用户（默认密码12345678）、删除用户、更新用户、按ID查询、分页查询
- 个人信息修改**：当前用户可修改自己的昵称、头像、简介
- 角色权限**：user（普通用户）/ admin（管理员）/ ban（封禁），通过 @AuthCheck 注解实现接口级权限控制

功能模块树

```
用户模块
├── 认证
│   ├── 用户注册 (POST /user/register)
│   ├── 用户登录 (POST /user/login)
│   ├── 用户登出 (POST /user/logout)
│   └── 获取当前登录用户 (GET /user/get/login)
├── 用户管理 (管理员)
│   ├── 创建用户 (POST /user/add)
│   ├── 删除用户 (POST /user/delete)
│   ├── 更新用户 (POST /user/update)
│   ├── 按ID查询 (GET /user/get)
│   ├── 按ID查询脱敏 (GET /user/get/vo)
│   ├── 分页查询 (POST /user/list/page)
│   └── 分页查询脱敏 (POST /user/list/page/vo)
└── 个人信息
    └── 修改个人信息 (POST /user/update/my)
```

流程图

用户注册流程

```
POST /user/register
|
▼
参数校验:
├── 非空校验 (userAccount/userPassword/checkPassword)
├── 账号长度 >= 4
├── 密码长度 >= 8
└── 两次密码一致
|
▼
账号去重 (synchronized 加锁):
├── 账号已存在 → 抛出异常 "账号重复"
└── 账号不存在 → 继续
|
▼
密码加密: MD5(SALT + userPassword), SALT="limou"
|
▼
插入 user 表 → 返回 userId
```

用户登录流程

```
POST /user/login
|
▼
参数校验 (非空、长度)
|
▼
密码加密: MD5(SALT + userPassword)
|
▼
```

```
查询 user WHERE userAccount = ? AND userPassword = ?
└─ 未找到 → 尝试按账号查找用户（记录登录失败日志）
    |      → 如有用户则记录 isSuccess=0, failReason="密码错误"
    |      → 抛出异常 "用户不存在或密码错误"
└─ 找到 → session.setAttribute("user_login", user)
        → 记录登录成功日志（isSuccess=1）
        → 返回 LoginUserVO（脱敏）
```

数据库设计

用户表 user

使用 MyBatis-Plus 雪花算法生成主键 (IdType.ASSIGN_ID)，字段采用 camelCase 命名。

字段	类型	说明
id	BIGINT (ASSIGN_ID)	主键，雪花ID
userAccount	VARCHAR	登录账号
userPassword	VARCHAR	MD5加密密码 (SALT="limou")
userName	VARCHAR	用户昵称
userAvatar	VARCHAR	用户头像URL
userProfile	VARCHAR	用户简介
userRole	VARCHAR	角色：user / admin / ban
roleId	BIGINT	角色ID（预留）
employeeId	BIGINT	关联员工ID
createTime	DATETIME	创建时间
updateTime	DATETIME	更新时间
isDelete	TINYINT	逻辑删除：0=否 1=是

实体类与DTO设计

User（用户实体）

字段	类型	说明
id	Long (ASSIGN_ID)	主键
userAccount	String	账号
userPassword	String	密码（MD5加密）
userName	String	昵称
userAvatar	String	头像URL
userProfile	String	简介
userRole	String	角色：user/admin/ban
roleId	Long	角色ID
employeeId	Long	员工ID
createTime	Date	创建时间
updateTime	Date	更新时间
isDelete	Integer	逻辑删除

LoginUserVO（登录用户视图，脱敏）

字段	类型	说明
id	Long	用户ID
userName	String	昵称
userAvatar	String	头像URL
userProfile	String	简介
userRole	String	角色
createTime	Date	创建时间
updateTime	Date	更新时间

注意：LoginUserVO 不包含 userAccount、userPassword、isDelete 等敏感字段。

UserVO（用户视图，脱敏）

字段	类型	说明
id	Long	用户ID
userName	String	昵称
userAvatar	String	头像URL
userProfile	String	简介
userRole	String	角色
createTime	Date	创建时间

API 设计

基础路径: `/user`，服务 context-path: `/api`，完整路径: `/api/user/*`

1. 用户注册

POST `/user/register`

请求参数 (UserRegisterRequest):

参数	类型	必填	描述
userAccount	String	是	账号 (>=4位)
userPassword	String	是	密码 (>=8位)
checkPassword	String	是	确认密码 (需与密码一致)

校验规则:

- 参数非空
- `userAccount.length() >= 4`
- `userPassword.length() >= 8`
- `userPassword.equals(checkPassword)`
- 账号不重复 (synchronized 加锁查询)

响应: `BaseResponse<Long>` — 返回新用户ID

2. 用户登录

POST `/user/login`

请求参数 (UserLoginRequest):

参数	类型	必填	描述
userAccount	String	是	账号
userPassword	String	是	密码

响应 (LoginUserVO):

```
{
  "code": 0,
  "data": {
    "id": 2075829151662010370,
    "userName": "张三",
    "userAvatar": "https://...",
    "userProfile": "前端工程师",
    "userRole": "user",
    "createTime": "2026-01-01T00:00:00",
    "updateTime": "2026-07-10T12:00:00"
  }
}
```

后端行为:

- 登录成功: session 存储 User 对象 (key="user_login") , 记录登录日志
- 登录失败: 若账号存在则记录失败日志 (failReason="密码错误")

3. 用户登出

```
POST /user/logout
```

说明: 清除 session 中 "user_login" 属性

响应: BaseResponse<Boolean> — true

4. 获取当前登录用户

```
GET /user/get/login
```

说明: 从 session 读取当前用户, 查询数据库获取最新信息, 返回脱敏后的 LoginUserVO

5. 用户管理 (管理员, 需 @AuthCheck(mustRole = "admin"))

创建用户

```
POST /user/add
```

请求参数 (UserAddRequest):

参数	类型	必填	描述
userName	String	否	昵称
userAccount	String	是	账号
userAvatar	String	否	头像URL
userRole	String	否	角色: user/admin

默认密码: 12345678 (MD5 加密后存储)

删除用户

```
POST /user/delete { "id": 123 }
```

更新用户

```
POST /user/update
```

请求参数 (UserUpdateRequest):

参数	类型	必填	描述
id	Long	是	用户ID
userName	String	否	昵称
userAvatar	String	否	头像URL
userProfile	String	否	简介
userRole	String	否	角色: user/admin/ban

按ID查询

```
GET /user/get?id=123          # 返回完整 User 对象（管理员）
GET /user/get/vo?id=123      # 返回脱敏 UserVO
```

分页查询

```
POST /user/list/page          # 返回 Page<User>（管理员）
POST /user/list/page/vo      # 返回 Page<UserVO>（限制 pageSize <= 20）
```

请求参数 (UserQueryRequest 继承 PageRequest):

参数	类型	必填	描述
current	long	是	页码
pageSize	long	是	每页条数 (/vo 接口限制 <=20)
id	Long	否	用户ID精确查询
userName	String	否	昵称模糊查询
userProfile	String	否	简介模糊查询
userRole	String	否	角色精确查询
unionId	String	否	微信unionId精确查询
mpOpenId	String	否	公众号openId精确查询
sortField	String	否	排序字段
sortOrder	String	否	排序方向：asc/desc

6. 修改个人信息

POST /user/update/my

请求参数 (UserUpdateMyRequest):

参数	类型	必填	描述
userName	String	否	昵称
userAvatar	String	否	头像URL
userProfile	String	否	简介

说明: 仅允许修改自己的信息, 不可修改账号、密码、角色。从 session 自动获取当前用户ID。

关键技术设计

密码加密方案

```
public static final String SALT = "limou";

// 加密
String encryptPassword = DigestUtils.md5DigestAsHex((SALT +
password).getBytes());
```

使用 MD5 + 固定 SALT。所有密码相关操作（注册、登录、修改密码）统一使用此方案。

Session 登录态

```
// 登录时存储
request.getSession().setAttribute("user_login", user);

// 获取当前用户
User currentUser = (User) request.getSession().getAttribute("user_login");
if (currentUser == null || currentUser.getId() == null) {
    throw new BusinessException(ErrorCode.NOT_LOGIN_ERROR);
}

// 从数据库重新查询确保数据最新
currentUser = this.getById(currentUser.getId());

// 登出时清除
request.getSession().removeAttribute("user_login");
```

角色权限控制

通过 `@AuthCheck` 注解 + AOP 切面实现接口级权限校验：

```
// UserConstant.java
String DEFAULT_ROLE = "user";
String ADMIN_ROLE = "admin";
String BAN_ROLE = "ban";

// 使用示例
@PostMapping("/add")
@AuthCheck(mustRole = UserConstant.ADMIN_ROLE)
public BaseResponse<Long> addUser(...) { ... }
```

注册账号去重

使用 `synchronized (userAccount.intern())` 对账号字符串加锁，防止并发注册同账号：

```
synchronized (userAccount.intern()) {
    long count = this.baseMapper.selectCount(queryWrapper);
    if (count > 0) {
        throw new BusinessException(ErrorCode.PARAMS_ERROR, "账号重复");
    }
    // ... 插入
}
```

登录日志记录

在 `userLogin()` 方法中直接调用 `LoginLogService.recordLoginLog()`，成功和失败均记录：

```
// 成功
loginLogService.recordLoginLog(user.getId(), request.getRemoteAddr(),
    request.getHeader("User-Agent"), 1, true, null);

// 失败（账号存在但密码错误）
loginLogService.recordLoginLog(existUser.getId(), request.getRemoteAddr(),
    request.getHeader("User-Agent"), 1, false, "密码错误");
```

脱敏处理

`getLoginUserVO()` 和 `getUserVO()` 通过 `BeanUtils.copyProperties()` 从 `User` 拷贝到 `VO`，自动排除 `User` 中独有的敏感字段（`userAccount`、`userPassword`、`isDelete`）。前者比后者多一个 `updateTime` 字段。

排期

阶段	内容	预估工期
需求评审	评审用户角色权限体系	0.5天
技术方案	确认认证方案、密码策略、session管理	0.5天
数据库开发	user 表创建与初始化	0.5天
后端开发	注册/登录/登出、用户CRUD、权限AOP、登录日志	2天
前端开发	登录页、注册页、用户管理页、个人设置页	2天
联调测试	认证流程测试、权限校验测试	1天

总预估工期：约 6.5 个工作日